金融數據分析網頁工具開發

書面報告

學號： **H34104028**

姓名：陳乙慈

目錄

作業 1. 爆大量 4

1.1 作業目標 4

1.2 實作中學習到的知識與遭遇的困難 4

1.3 結果截圖 5

1.4 程式使用方式 6

作業 2. 股票定價策略 7

2.1 作業目標 7

2.2 實作中學習到的知識與遭遇的困難 7

2.3 結果截圖 7

2.4 程式使用方式 10

作業 3. 本益比河流圖 11

3.1 作業目標 11

3.2 實作中學習到的知識與遭遇的困難 11

3.3 結果截圖 11

3.4 程式使用方式 12

作業 4. 天花板地板線 13

4.1 作業目標 13

4.2 實作中學習到的知識與遭遇的困難 13

4.3 結果截圖 13

4.4 程式使用方式 14

作業 5. 技術指標進出場 15

5.1 作業目標 15

5.2 實作中學習到的知識與遭遇的困難 15
5.3 結果截圖 15
5.4 程式使用方式 17
作業 6. 帳號密碼功能 19
6.1 作業目標 19
6.2 實作中學習到的知識與遭遇的困難 19
6.3 結果截圖 19
6.4 程式執行方法 20
作業 7. Tracker 功能、Email 功能 21
7.1 作業目標 21
7.2 實作中學習到的知識與遭遇的困難 21
7.3 結果截圖 22
7.4 程式執行方式 22
作業 8. Docker 封裝 23
8.1 作業目標 23
8.2 實作中學習到的知識與遭遇的困難 23
8.3 結果截圖 24
8.4 程式執行方式 24

作業 1. 爆大量

1.1 作業目標

第一天的作業目標是開發一個基於網頁的股票成交量分析系統，透過動態查詢與均線分析來識別「爆大量」現象，幫助使用者判斷市場熱度與流動性風險。為了完成作業要求，我們將透過 yfinance 取得歷史股市數據，並計算成交量的五日移動均線以判斷成交量是否出現「爆大量」，最後透過 HighCharts 和 DataTable 進行視覺化呈現。

1.2 實作中學習到的知識與遭遇的困難

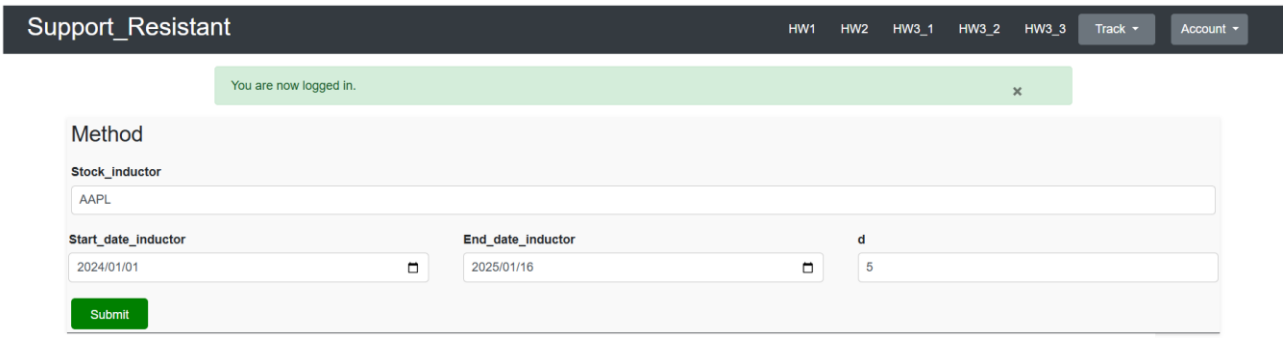
這次的作業是我第一次接觸網頁後端開發，以前我只有學習過 Python 語法以及網頁前端的基本概念（HTML、CSS、JavaScript）。雖然在課程開始前，有先練習過教授提供的加法器應用練習，但對於 Django 框架仍然非常陌生。在這次作業中，我以加法器的練習過程為基礎，循序漸進地完成開發，從新增一個 Django app 開始，逐步實作各個功能。

由於缺乏經驗，我花了大量時間在解決前後端資料傳輸的問題。例如，從前端 form 裡輸入的資料 (input) 無法順利被後端接收，或是後端透過 yfinance 抓取的金融數據無法正確傳遞至前端顯示。這些問題促使我深入理解 Django 的 views.py 與 hw1.js 之間的互動，並學習如何透過 GET 方法來正確處理請求與回應。

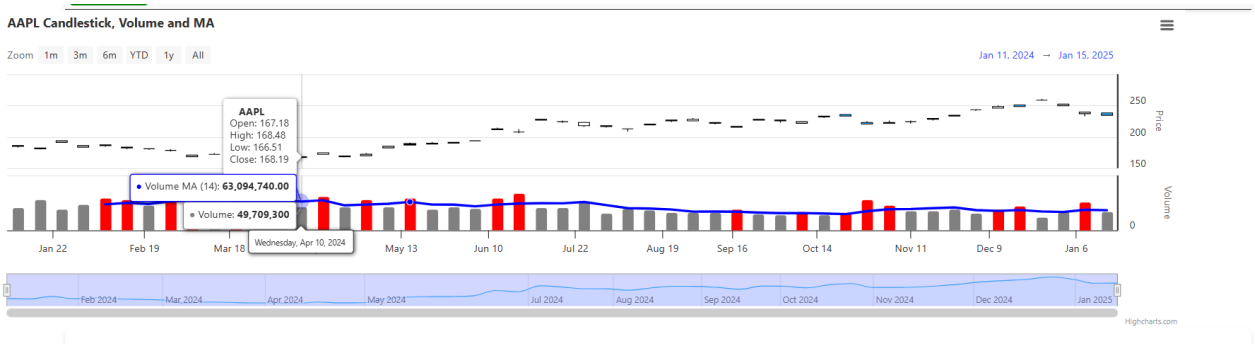
在解決資料傳輸問題後，我嘗試將獲取的資料視覺化。然而，因為對 High Charts 的不熟悉，我花了許多時間研究該如何調整設定來改變圖表呈現效果，確保資料能夠清晰且符合作業要求的呈現出來。這次作業完成並理解後，會發現基本上並不困難，只要理解 Django 前後端的邏輯，便可以順利完成。

備註：這次作業中，我還遇到了一個特別的問題，主要出現在時間戳的處理上。時間戳有的是以毫秒為單位表示，有的則以秒為單位，這樣的差異導致時間戳的數值不一致，進而影響時間轉換的正確性。在將時間戳轉換為年月日格式時，我便因為這點遇到多次錯誤，導致資料顯示異常。我最終也是意外發現，才成功解決此問題。

1.3 結果截圖



使用者輸入



蠟燭圖(含開高低收、五日均線)

Date	High Volume	Five Day MA
2024-02-05	69,668,800	56,892,400
2024-02-15	65,434,500	60,153,620
2024-03-01	73,488,000	63,544,940
2024-03-06	68,587,700	66,168,920
2024-03-21	106,181,300	72,412,420
2024-04-15	73,531,800	65,773,200
2024-04-30	65,934,800	56,357,880
2024-05-15	70,400,000	62,236,880
2024-06-14	70,122,700	56,460,660
2024-06-24	80,727,000	58,526,060
2024-09-17	45,519,300	40,780,460
2024-10-07	39,505,400	37,885,900
2024-10-22	38,846,600	35,650,480
2024-11-01	65,276,700	41,900,620
2024-11-06	54,561,100	46,236,720

列出發生爆大量日期的表格

1.4 程式使用方式

資料夾名稱：HW1

執行方式：(需先安裝 requirement.txt 中套件)

1. python manage.py runserver
2. 開啟瀏覽器並輸入 127.0.0.1:8000
3. 輸入股票代號、起始結束日期及 d 值並點擊 Submit

作業 2. 股票定價策略

2.1 作業目標

透過四種股票定價方法（股利法、高低價法、本淨比法、本益比法）來評估特定股票的投資價值（用公式產生對於此股票的便宜價、合理價、昂貴價，與此時價格相比，判斷是否進場。）

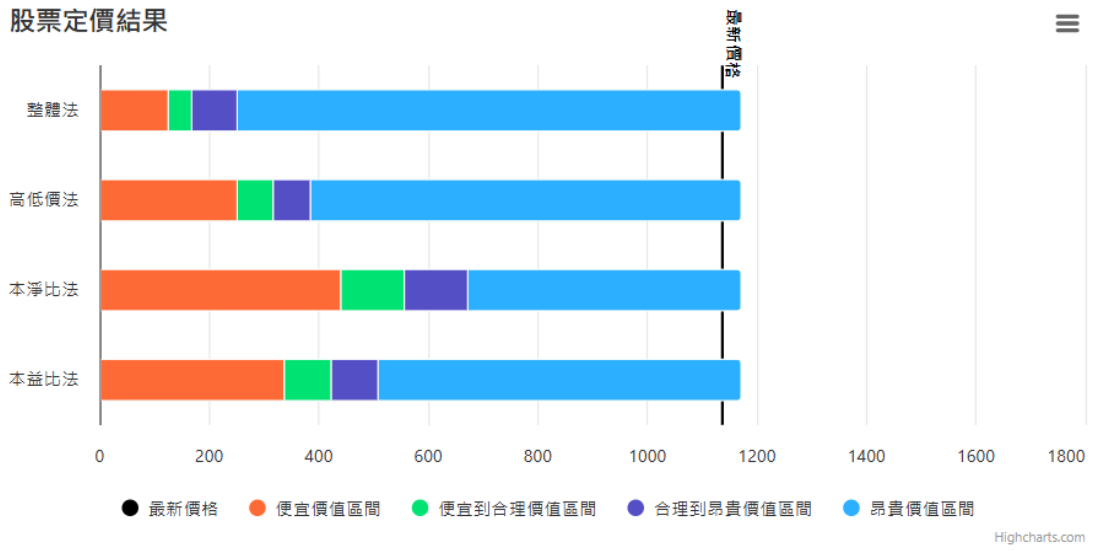
2.2 實作中學習到的知識與遭遇的困難

經過第一天的練習後，我對 Django 和 High Charts 的應用有了更深入的理解，因此在第二天的使用上明顯更加順手。然而，第二天真正讓我感到困難的部分並不是框架的應用，而是「資料蒐集」。因為我之前沒有使用爬蟲的經驗，因此在開始實作前，我先上網查閱了相關教學，學習如何使用 Selenium 來爬取動態網站的資料。

在資料蒐集的過程中，我使用 Selenium 來自動化瀏覽「Goodinfo！台灣股市資訊網」，透過選擇適當的選單項目並抓取網頁中的表格數據，來獲取歷年股利政策、本益比（PER）、股價淨值比（PBR）等資訊。爬取的資料再經過 Pandas 進行清理與整理，最後存成 CSV 檔案，方便後續分析與計算。

2.3 結果截圖

使用者輸入



股票定價結果圖

股利法 計算數據

透過當期股利或歷史平均股利乘上合理的回本時間估算股價。

股價 = 平均 10 年股利 x 你所希望的回本期間 (以年為單位)

便宜價 = 平均 10 年股利 x 15年 = 126

合理價 = 平均 10 年股利 x 20年 = 168

昂貴價 = 平均 10 年股利 x 30年 = 252

評價: 目前價格(1137.5) > 便宜價(126)

股利發放年度	現金股利盈餘	現金股利公積	現金股利合計	股票股利盈餘	股票股利公積	股票股利合計	股利合計
2023	11.5	0	11.5	0	0	0	11.5
2022	11	0	11	0	0	0	11
2021	10.5	0	10.5	0	0	0	10.5
2020	10	0	10	0	0	0	10
2019	12.5	0	12.5	0	0	0	12.5
2018	8	0	8	0	0	0	8
2017	7	0	7	0	0	0	7
2016	6	0	6	0	0	0	6
2015	4.5	0	4.5	0	0	0	4.5
2014	3	0	3	0	0	0	3

股利法 計算數據

高低價法 計算數據

透過歷史股價的平均估算股價。

股價 = 平均 10 年股價

便宜價 = 近 10 年每年最低股價平均 = 250.55

合理價 = 近 10 年每年平均股價平均 = 317.4

昂貴價 = 近 10 年每年最高股價平均 = 383.9

評價: 目前價格(1137.5) > 便宜價(250.55)

年度	最高	最低	收盤	平均
2023	594	443	593	543
2022	688	370	448.5	516
2021	679	518	615	598
2020	530	235.5	530	379
2019	345	206.5	331	262
2018	268	210	225.5	237
2017	245	179	229.5	210
2016	193	130.5	181.5	166
2015	155	112.5	143	140
2014	142	100.5	141	123

高低價法 計算數據

本淨比法 計算數據

透過歷史平均PBR乘上最新BPS估算股價

股價 = 近 10 年PBR平均 * 最新淨值

便宜價 = 近 10 年最低PBR平均 * 最新淨值 = 440.02062

合理價 = 近 10 年平均PBR平均 * 最新淨值 = 555.12756

昂貴價 = 近 10 年最高PBR平均 * 最新淨值 = 670.9013999999999

評價: 目前價格(1137.5) > 便宜價(440.02062)

年度	BPS(元)	最高PBR	最低PBR	平均PBR
2023	133.38	4.45	3.32	4.07
2022	113.6	6.06	3.26	4.54
2021	83.62	8.12	6.19	7.15
2020	71.33	7.43	3.3	5.31
2019	62.53	5.52	3.3	4.19
2018	64.67	4.14	3.25	3.67
2017	58.7	4.17	3.05	3.58
2016	53.58	3.6	2.44	3.1
2015	47.11	3.29	2.39	2.97
2014	40.32	3.52	2.49	3.04

本淨比法 計算數據

本益比法 計算數據

透過近一年EPS和歷史平均EPS的平均乘上歷史平均PER估算股價

股價 = ((近一年EPS + 近 10 年平均EPS) / 2) * 近 10 年PER平均

便宜價 = ((近一年EPS + 近 10 年平均EPS) / 2) * 近 10 年最低PER平均 = 337.05223499999994

合理價 = ((近一年EPS + 近 10 年平均EPS) / 2) * 近 10 年平均PER平均 = 422.3731500000001

昂貴價 = ((近一年EPS + 近 10 年平均EPS) / 2) * 近 10 年最高PER平均 = 508.2839000000001

評價: 目前價格(1137.5) > 便宜價(337.05223499999994)

年度	EPS(元)	最高PER	最低PER	平均PER
2023	32.34	18.4	13.7	16.8
2022	39.2	17.6	9.44	13.2
2021	23.01	29.5	22.5	26
2020	19.97	26.5	11.8	19
2019	13.32	25.9	15.5	19.6
2018	13.54	19.8	15.5	17.5
2017	13.23	18.5	13.5	15.9
2016	12.89	15	10.1	12.9
2015	11.82	13.1	9.52	11.8
2014	10.18	13.9	9.87	12

本益比法 計算數據

2.4 程式使用方式

資料夾名稱：HW1

執行方式：(需先安裝 requirement.txt 中套件)

1. python manage.py runserver
2. 開啟瀏覽器並輸入 127.0.0.1:8000
3. 輸入股票代號、歷史幾年資料並點擊 Submit

作業 3. 本益比河流圖

3.1 作業目標

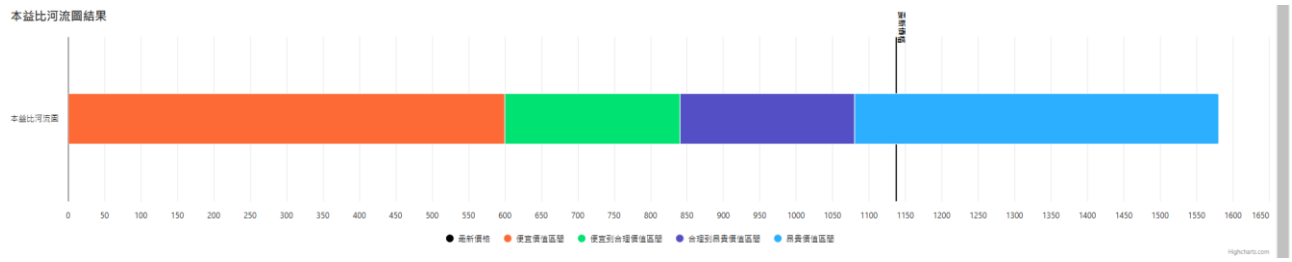
使用本益比河流圖（PER River Chart）來評估特定股票的股價區間

3.2 實作中學習到的知識與遭遇的困難

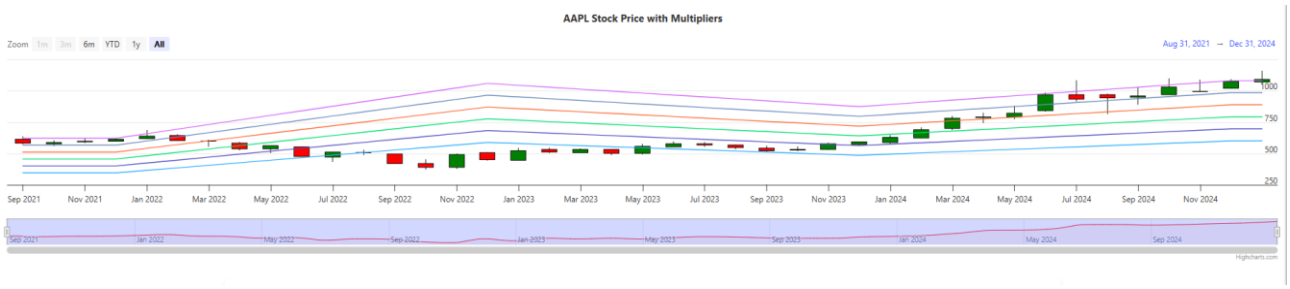
因為有作業二的基礎，對於本益比的計算方法以及爬蟲技術已經有一定的理解，因此這次的作業基本上並未花費太多時間便順利完成，過程中也並未遇到太多困難。不過在這次作業裡我有遇到一個特別的小問題，當我從 high chart 中把想要的圖表程式碼抓下來後，卻無法正常地在網頁中呈現，後來詢問過助教才知道，因為我過往都習慣點擊網頁左上角的重整鍵，但其實這會有一定的問題，導致網頁沒有真正的被重整，因此，此時應使用 Ctrl＋Shift＋R 才是最保險且有用的。

3.3 結果截圖

使用者輸入



本益比河流圖



K 線圖

本益比 表格結果

3.4 程式使用方式

資料夾名稱：HW1

執行方式：(需先安裝 requirement.txt 中套件)

1. python manage.py runserver
2. 開啟瀏覽器並輸入 127.0.0.1:8000
3. 輸入股票代號以及範圍並點擊 Submit

作業 4. 天花板地板線

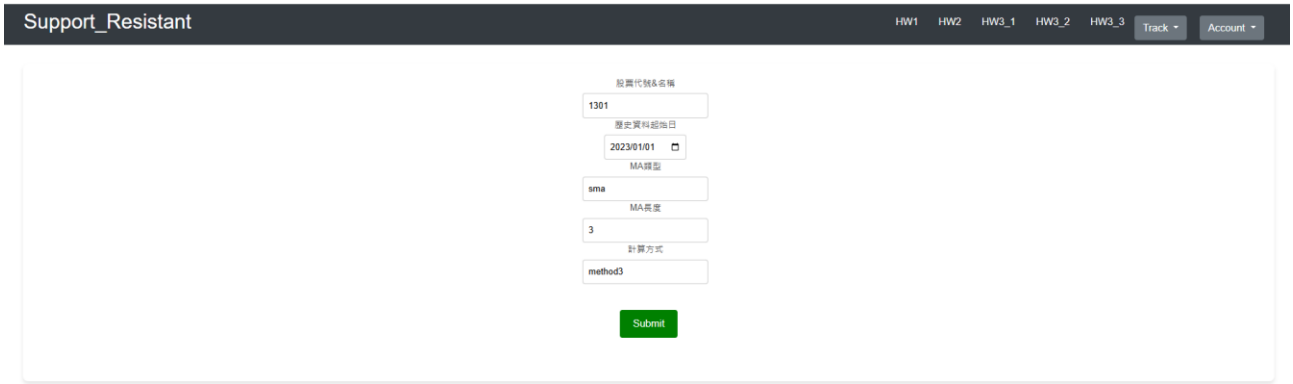
4.1 作業目標

計算股票的天花板線和地板線，並透過不同的方法來確定這些線的位置，以分析股價變動的極端範圍。

4.2 實作中學習到的知識與遭遇的困難

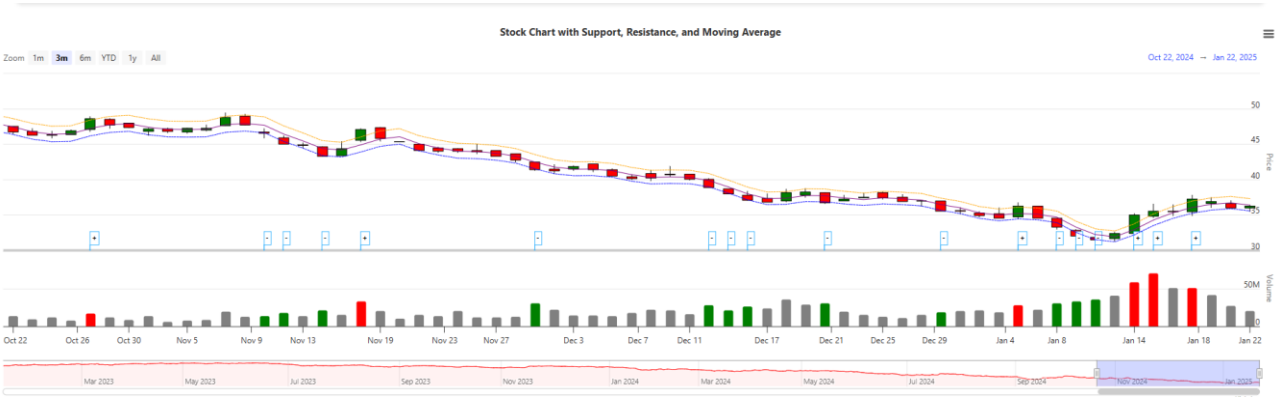
這次的作業我認為並不困難，但相當繁瑣麻煩。主要因為我對這些金融數據分析的計算方式不夠熟悉，所以花了一些時間去理解它們以及這些數學計算方法。而剩下的資料抓取和圖表呈現的部分跟先前作業基本大同小異，因此處理起來算是簡單。

4.3 結果截圖

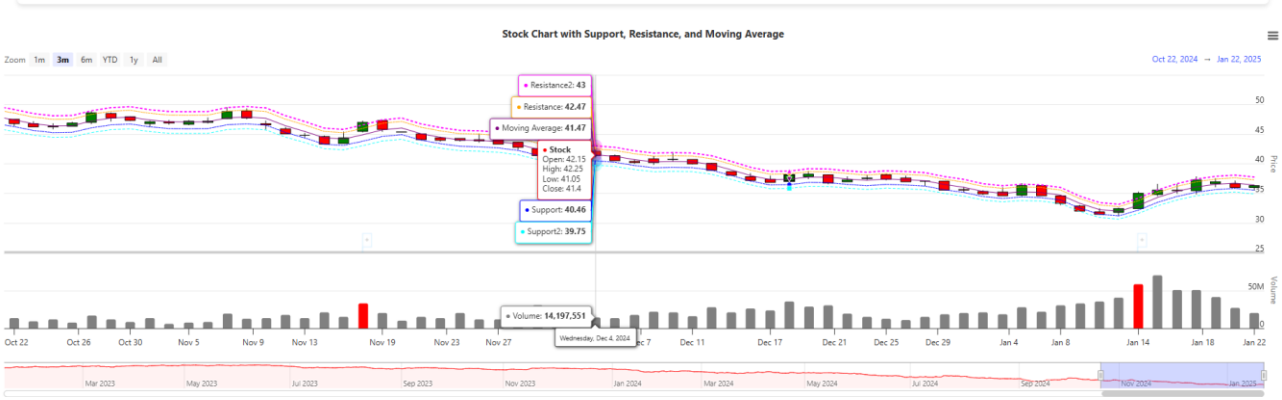


使用者輸入

Method 1



Method 2



Method 3

4.4 程式使用方式

資料夾名稱：HW1

執行方式：(需先安裝所有子資料夾中的 requirement.txt 中套件)

1. python manage.py runserver
2. 開啟瀏覽器並輸入 127.0.0.1:8000
3. 輸入股票代號、歷史資料起始日、MA 類型、長度以及計算方式並點擊 Submit

作業 5. 技術指標進出場

5.1 作業目標

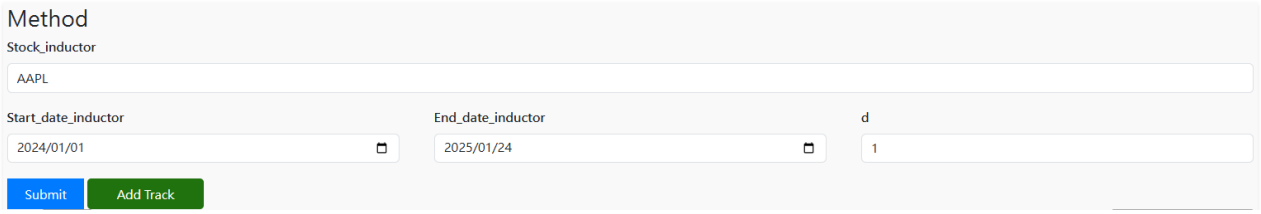
此作業是透過各種技術指標來分析金融市場走勢，並進行市場趨勢判斷。

5.2 實作中學習到的知識與遭遇的困難

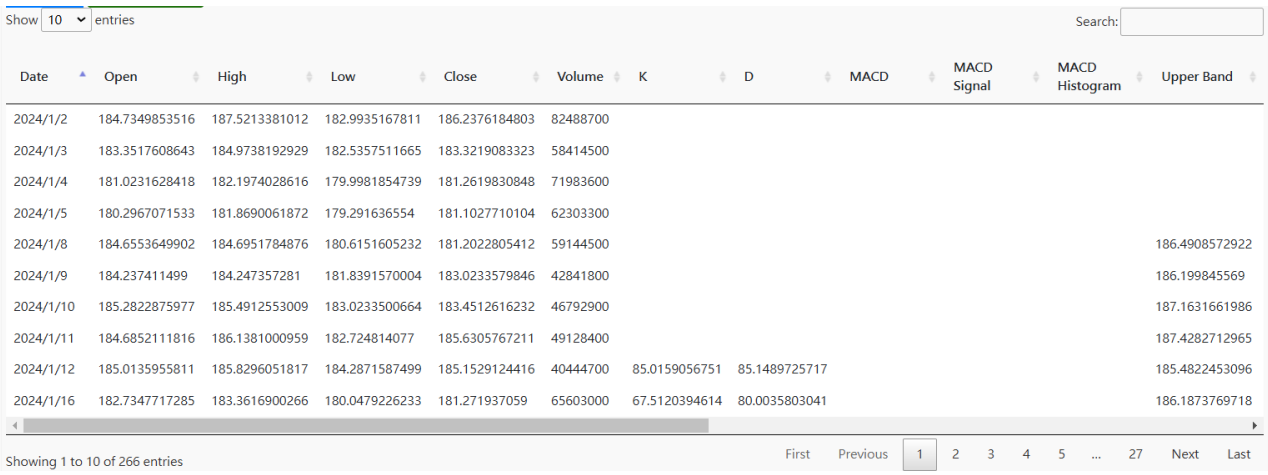
此次作業是透過 KD 指標、MACD 指標、布林通道等技術分析工具來預測金融市場的走勢，並標記潛在的買賣訊號。困難點在於對這些技術指標的不了解，資料的抓取跟圖表呈現基本上和先前作業大同小異，只是需要特別去計算並在圖中標記出買賣訊號的發生，這一部分是真的特別麻煩。

整體而言，作業三、四、五其實都是第三天的作業，雖然技術上並非非常困難，但有許多小細節需要注意，實作上十分繁瑣，因此第三天的壓力其實也不小。

5.3 結果截圖



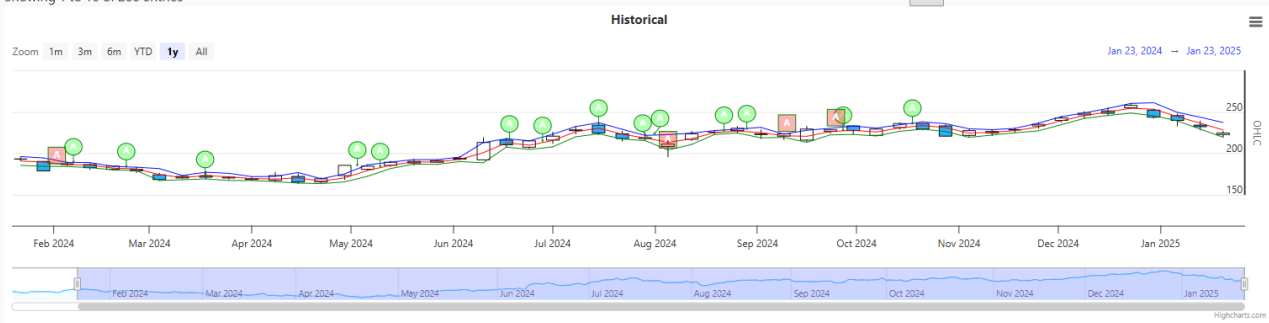
使用者輸入

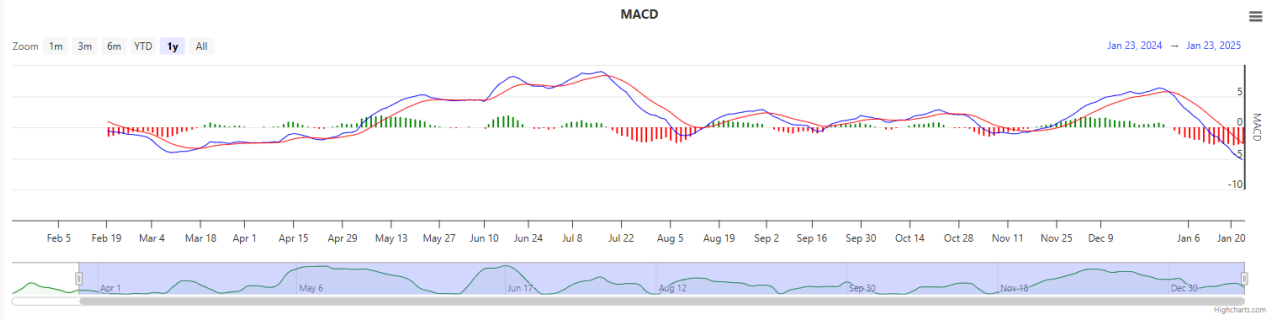


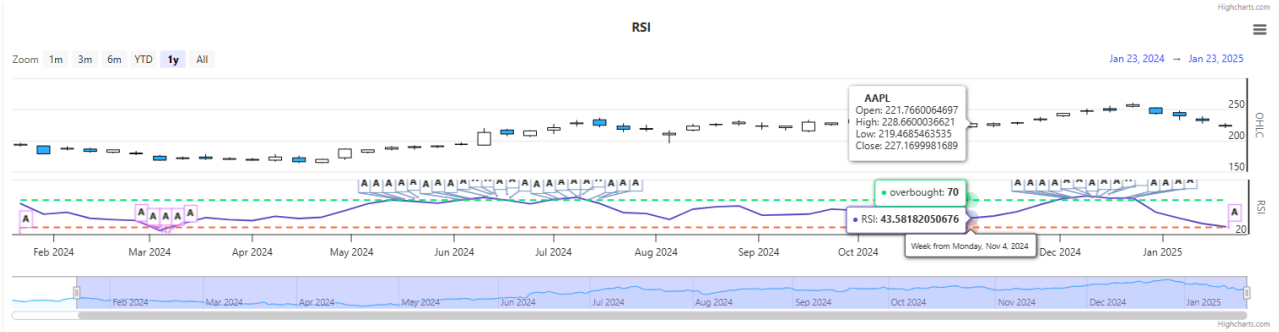
Date	Open	High	Low	Close	Volume	K	D	MACD	MACD Signal	MACD Histogram	Upper Band
2024/1/2	184.7349853516	187.5213381012	182.9935167811	186.2376184803	82488700						
2024/1/3	183.3517608643	184.9738192929	182.5357511665	183.3219083323	58414500						
2024/1/4	181.0231628418	182.1974028616	179.9981854739	181.2619830848	71983600						
2024/1/5	180.2967071533	181.8690061872	179.291636554	181.1027710104	62303300						
2024/1/8	184.6553649902	184.6951784876	180.6151605232	181.2022805412	59144500						186.4908572922
2024/1/9	184.237411499	184.247357281	181.8391570004	183.0233579846	42841800						186.199845569
2024/1/10	185.2822875977	185.4912553009	183.0233500664	183.4512616232	46792900						187.1631661986
2024/1/11	184.6852111816	186.1381000959	182.724814077	185.6305767211	49128400						187.4282712965
2024/1/12	185.0135955811	185.8296051817	184.2871587499	185.1529124416	40444700	85.0159056751	85.1489725717				185.4822453096
2024/1/16	182.7347717285	183.3616900266	180.0479226233	181.271937059	65603000	67.5120394614	80.0035803041				186.1873769718

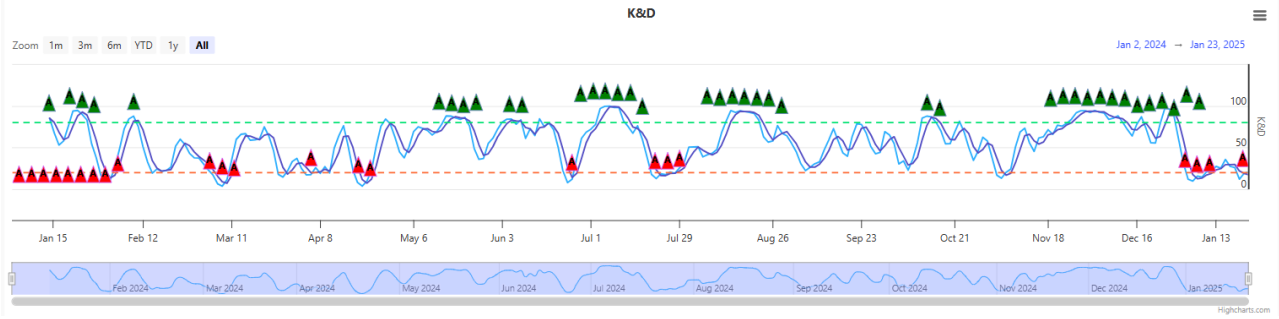
Showing 1 to 10 of 266 entries

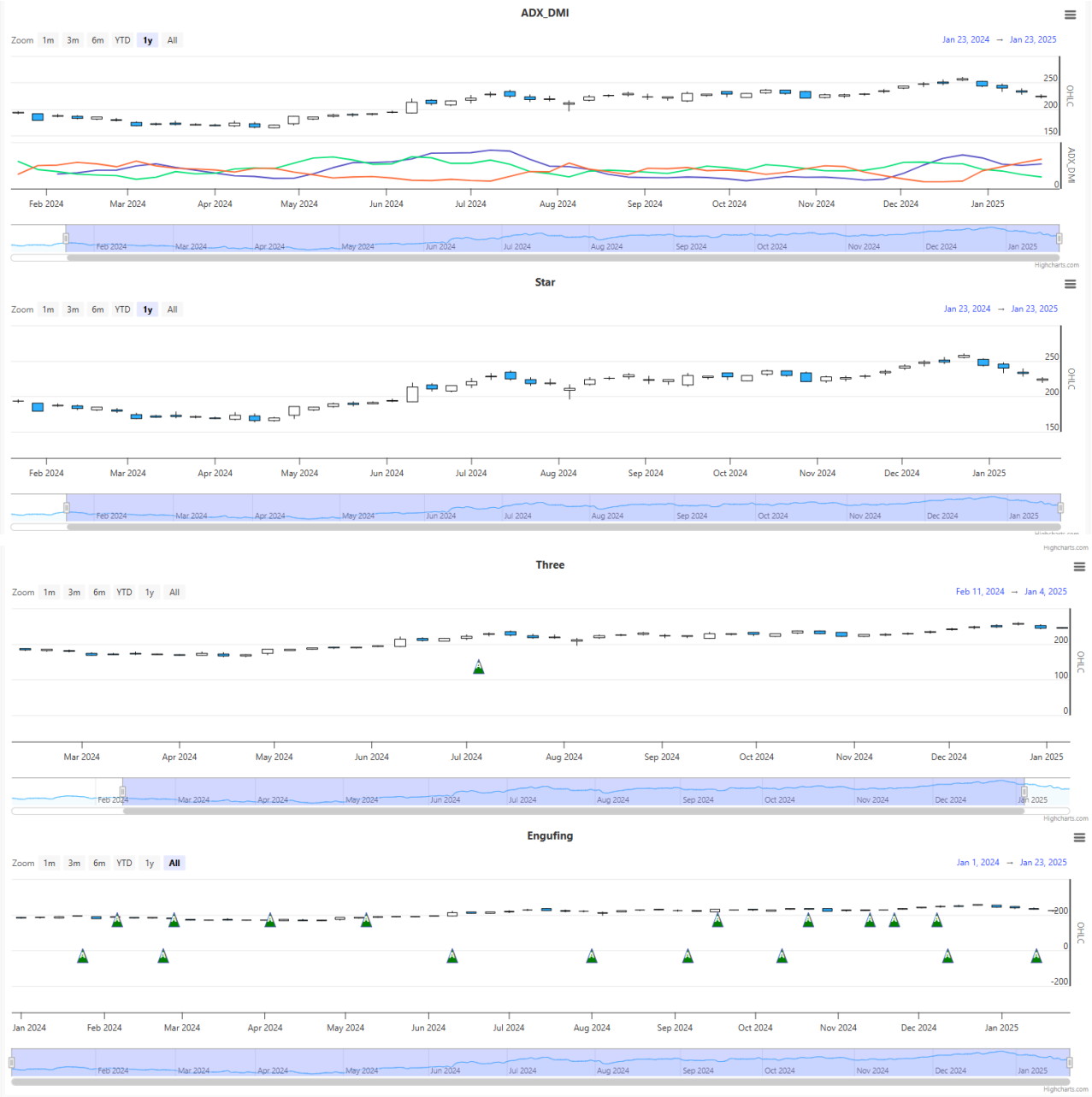
First Previous 1 2 3 4 5 ... 27 Next Last











各技術指標之圖表呈現

5.4 程式使用方式

資料夾名稱：HW1

執行方式：(需先安裝所有子資料夾中的 requirement.txt 中套件)

1. python manage.py runserver
2. 開啟瀏覽器並輸入 127.0.0.1:8000

3. 輸入股票代號、起始結束日以及 d 值並點擊 Submit

作業 6. 帳號密碼功能

6.1 作業目標

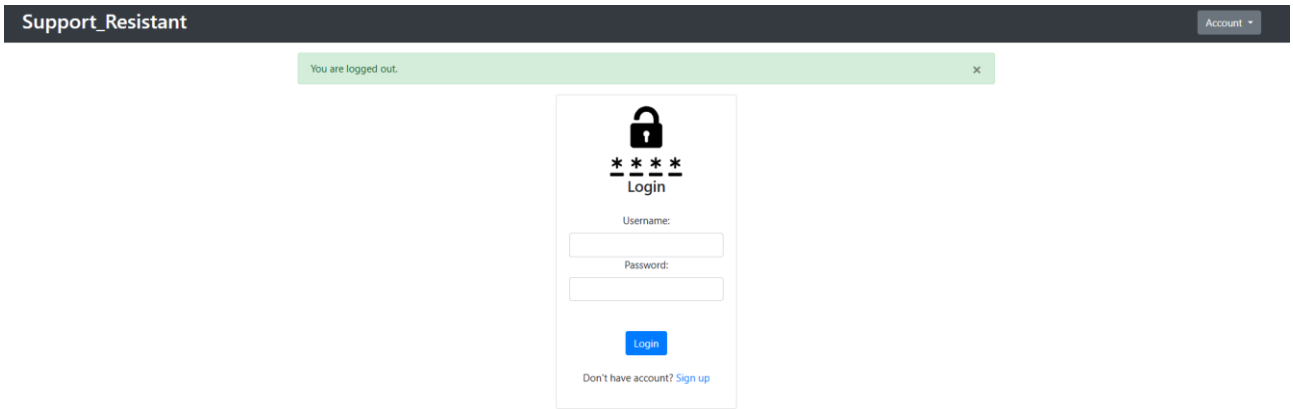
執行使用提供的程式碼在本地端創建帳號密碼的資料表，修改程式碼使登入及註冊頁面順利運作並在使用者未登入的情況下將其強制導入登入頁面。

6.2 實作中學習到的知識與遭遇的困難

這次的作業是實作網站的帳號密碼功能。當時助教講解完作業要求後，我對於這個作業真的是一頭霧水，因為前三天的作業彼此之間都有相似之處，實作起來比較直覺，可以逐步完成。然而，這次作業涉及全新的帳號密碼功能，讓我一時之間不知該如何下手。

後來，在助教的額外教學與指導下，我才逐步理解整個流程，從安裝 PostgreSQL 到修改設定檔資訊，以及如何正確導引網址路徑。不過我在導引網址路徑的部分，也花了許多心力，因為不了解 Django 內是如何做到網址間導引以及呼叫到不同頁面的，因此花了許多時間向助教學習及試錯，最終才順利完成。(學習到 path('HW1/', include('HW1.urls'))的用法)

6.3 結果截圖



登入頁面

6.4 程式執行方法

資料夾名稱：accounts、HW1

執行方式：

1. 打開任意網頁若在未登入狀態下，皆會先被導引到登入頁面，此時若沒有帳號，則需先註冊帳號。

作業 7. Tracker 功能、Email 功能

7.1 作業目標

Tracker 功能，讓使用者可以選擇自己感興趣的交易資訊並將其加入追蹤清單，以利未來進行後續追蹤及分析，另外當進行技術指標分析時，會將結果(有無訊號產生)傳送到使用者的 email 中。

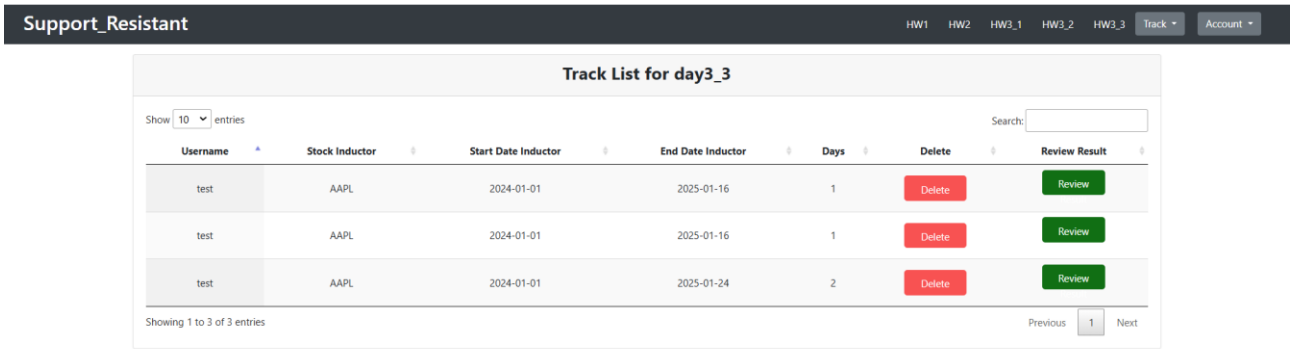
7.2 實作中學習到的知識與遭遇的困難

這次的作業是在已有的帳號密碼功能基礎上，進一步實作不同使用者的 Tracker 系統。核心目標是透過 SQL 語法，結合前端操作與資料庫資料表的新增、刪除等動作，讓不同使用者能夠管理自己的追蹤清單。

在實作過程中，我認為最困難的部分是在理解資料表的運作方式，並學習如何使用 SQL 語法來正確處理追蹤資料的增刪查改。這部分不僅繁瑣，還需要特別注意資料的正確性與一致性。此外，前端的資料表呈現也讓我花費不少時間，因為需要確保使用者的追蹤資訊能夠即時顯示並動態更新。另外當在 tracker 表中，若想要 review 此筆資料，當點擊時會將資料重新送回技術指標的頁面，並將結束日期調整成當日，此時點擊 submit 即可進行分析。

Email 傳送功能則較為簡單，只要引入 email 的模組後，依照固定的程式碼及設定即可依據訊號是否發生，來寄出相應資訊到登入者的 Email 中(從 usr 的 table 中可以取得)

7.3 結果截圖



Tracker 頁面

7.4 程式執行方式

資料夾名稱：HW1

執行方式：

1. cd accounts
2. python manage.py runserver 127.0.0.1:8080
3. cd ..
4. cd NCKU
5. python manage.py runserver 127.0.0.1:8000

作業 8. Docker 封裝

8.1 作業目標

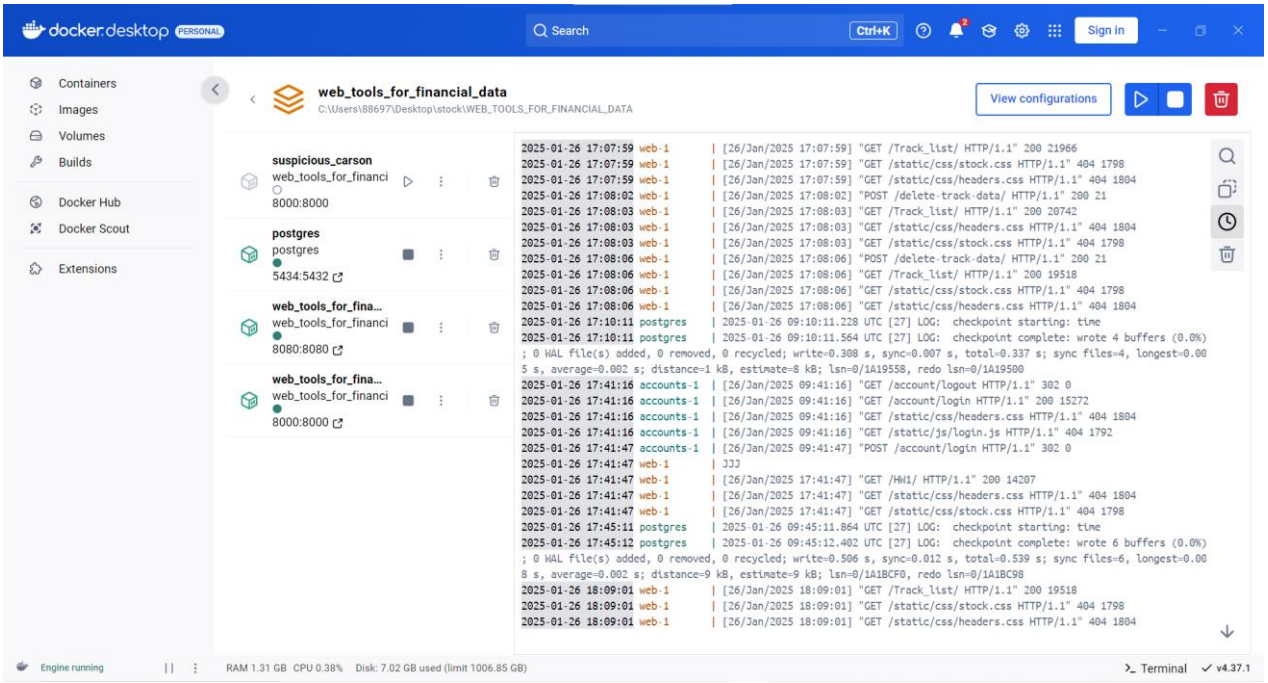
使用 Docker 來包裝程式，透過容器化技術將應用程式封裝成可移植的映像檔，以便在不同環境中部署與執行

8.2 實作中學習到的知識與遭遇的困難

這是我第一次接觸 Docker 這類容器化工具，在實作前，我先學習了容器化的相關概念(image, container , repository, 和 VM 的差別等)，以便後續能更順利進行實作。然而，概念相對容易理解，但實作起來卻困難重重，再加上對 Docker 語法的不熟悉，我花了許多時間在試錯與調整。後來，透過觀看多部教學影片，我才逐漸掌握 Docker 的基本脈絡，並發現可以使用 docker-compose.yaml 來設定多個容器，例如：資料庫（PostgreSQL）、Web 應用（Django）、以及帳號管理服務，確保這些服務之間能夠順利溝通。

在實作過程中，我也遇到了各種挑戰，例如 Docker 重啟後資料庫內的資料會遺失，以及 Web 應用無法連線至資料庫，導致存取錯誤。透過不斷的 try and error，我逐步理解了 Docker 的運作方式，成功完成這項作業，並對 Docker 的應用有了更深入的理解。

8.3 結果截圖



Docker 頁面

8.4 程式執行方式

資料夾名稱：accounts、HW1

執行方式：

2. docker-compose up --build -d

3. 開啟瀏覽器並輸入 127.0.0.1:8000/HW1/